95. Drone-Matrix: Courier Dispatch

DSA Final Project

Submitted By:

Name: Samarth Navale
Roll Number: 150096725148
Cohort: Sam Altman
Branch: B.Tech Computer Science Engineering
Semester: 2
Institute: ITM
Academic Year: 2025-2029

Abstract

Drone-Matrix is a C++-based airspace simulation and drone dispatch management system designed to model large-scale urban drone delivery operations. The system manages package dispatch, route planning, battery-aware drone allocation, congestion handling, emergency flight rollback, charging pad allocation, and real-time fleet tracking.

The project applies core Data Structures and Algorithms including Graphs, Dijkstra's Algorithm, Queues, Stacks, HashMaps, Priority Queues, Merge Sort, and Quick Sort to solve real-world logistics problems. A React-based visualization layer is additionally developed to demonstrate simulation states and operational workflows during project presentation.

Problem Statement (95. Courier Dispatch Airspace Drone Matrix)

A delivery company operates hundreds of drones across a city. Every day, thousands of packages must be delivered while avoiding restricted airspace zones, battery failures, congestion, and weather disruptions.

The system must:

- Enforce airspace restrictions
- Support emergency flight rollback
- Process package dispatch in FIFO order
- Resolve customer addresses rapidly
- Allocate drones efficiently
- Calculate optimal routes
- Manage charging pad assignments
- Track drone movement in real time

Objectives

Primary Objectives

- Build a drone dispatch simulation engine.
- Implement shortest-path route planning.
- Prevent restricted airspace violations.
- Track battery consumption and charging.
- Manage congestion and rerouting.
- Support emergency recovery mechanisms.

Secondary Objectives

- Visualize fleet activity.
- Demonstrate real-time simulation ticks.
- Present route progression and mission status.
- Provide an educational demonstration of DSA concepts.

Technology Stack

Backend

- C++
- STL Containers
- Object-Oriented Programming

Frontend Visualization

- React.js
- Vite
- SVG Graphics
- CSS

Data Storage

- Text Configuration Files

System Architecture

Data Files
↓
Manager Layer
↓
Services Layer
↓
Simulation Engine
↓
CLI Interface
↓
React Visualization Layer

Core Modules

Airspace Manager

Maintains nodes, corridors, traffic counts, and restricted zones.

Drone Manager

Maintains fleet information and drone allocation logic.

Package Manager

Maintains FIFO dispatch queue.

Flight Manager

Maintains rollback checkpoints.

Mission Manager

Tracks delivery missions and lifecycle states.

Pad Manager

Allocates charging pads and manages occupancy.

Route Service

Computes shortest routes and battery costs.

Address Service

Performs address-to-node mapping.

Simulator

Coordinates the complete simulation lifecycle.

Data Structures Used

Graph

Purpose:
Represents city airspace.

Implementation:
Adjacency List

Applications:

- Route Planning
- Restricted Zone Avoidance
- Congestion Handling

Queue

Purpose:
Package Dispatch

Implementation:
FIFO Queue

Operations:

- Enqueue Package
- Dequeue Package

Complexity:
O(1)

Stack

Purpose:
Flight Undo System

Implementation:
FlightState Stack

Operations:

- Push Checkpoint
- Pop Checkpoint

Complexity:
O(1)

HashMap

Purpose:
Address Lookup

Implementation:
unordered_map

Operations:

- Coordinate Lookup
- Node Lookup

Complexity:
O(1) Average

Algorithms Used

Dijkstra Algorithm

Purpose:
Shortest Route Generation

Why Used:
Airspace corridors contain weighted edges representing travel cost.

Complexity:
O((V + E) log V)

Merge Sort

Purpose:
Fleet Battery Sorting

Complexity:
O(N log N)

Quick Sort

Purpose:
Alternative Fleet Sorting

Complexity:
Average O(N log N)

Requirement Implementation

Airspace Rules

Restricted nodes are excluded during route generation.

Flight Undo

Flight checkpoints are stored using stacks and restored during emergencies.

Launch Line

Packages are dispatched using FIFO queue processing.

Address Lookup

Order IDs are mapped to coordinates using HashMaps.

Drone Sorter

Available drones are ranked according to battery margins.

Best Flight Path

Dijkstra computes optimal routes while avoiding restricted areas and congested corridors.

Pad Manager

The nearest available charging pad is selected using shortest-path calculations.

Simulation Engine

Mission States

ACTIVE

RETURNING

EMERGENCY

BLOCKED

DELIVERED

ARCHIVED

Battery Safety Model

Required Energy =

Pickup Cost
+
Delivery Cost
+
Safe Exit Cost

Safe Exit Cost =

min(Return Cost, Pad Cost)

Only drones with positive energy margin may be dispatched.

Congestion Management

Corridors contain:

- Capacity
- Current Traffic

When capacity is reached:

- Drone waits
- Reroute attempted
- Deadlock detection triggered

After 3 wait ticks:

- Forced reroute
- BLOCKED state if reroute fails

React Visualization Layer

Purpose:
Presentation and demonstration.

Features:

- Airspace Map
- Fleet Dashboard
- Package Queue
- Mission Panel
- Event Log
- Simulation Controls

Note:

The React application does not execute simulation logic. The authoritative implementation remains the C++ simulation engine.

```
[No-Fly Restricted Zones]: Node 11 (Restricted_Hospital) Node 4 (Restricted_Airport)

[Simulator Logs]:
  * Simulator initialized successfully.
================================================================================
                     COURIER DISPATCH AIRSPACE DRONE MATRIX
                       (Simulation Tick: 0)
================================================================================
 Fleet Size: 5 Drones | Warehouses: 1 | Charging Pads: 3 | Restricted Zones: 2 | Active Packages: 0
================================================================================
Active Flights: None
================================================================================
1. Airspace Overview
2. Fleet Dashboard
3. Package Queue
4. Address Lookup Demo
5. Dispatch Delivery
6. Advance One Tick
7. Advance Five Ticks
8. Trigger Storm
9. Undo Flight
10. Charging Pad Dashboard
11. Run Full Demonstration
12. Exit
================================================================================
Enter choice (1-12):
```

```
 ================================================
 LIVE FLEET STATUS
 ================================================
 Active Missions: 0
 Archived Missions: 0
 ------------------------------------------------
 Drone: 105
 Current Node: 0 (Warehouse_Alpha)
 Battery Remaining: 30
 Status: IDLE
 ------------------------------------------------
 Drone: 104
 Current Node: 2 (East_Corridor_1)
 Battery Remaining: 86
 Status: IDLE
 ------------------------------------------------
 Drone: 103
 Current Node: Pad 3
 Battery Remaining: 7
 Status: IDLE
 ------------------------------------------------
 Drone: 102
 Current Node: 0 (Warehouse_Alpha)
 Battery Remaining: 85
 Status: IDLE
 ------------------------------------------------
 Drone: 101
 Current Node: 1 (North_Corridor_1)
 Battery Remaining: 94
 Status: IDLE
 ------------------------------------------------

 Choose Fleet Reporting Sort:
 1. None (Default order)
 2. Merge Sort (Descending Battery)
 3. Quick Sort (Descending Battery)
 Enter sub-choice (1-3):
```

```
====================
TICK 1
====================
Drone 103
Charging at Pad 3
Battery: 7 -> 22
Status: CHARGING
--------------------

Traffic Integrity Check: PASSED
- No flight corridors exceeded capacity.
- No negative traffic counts detected.
- Reserve/Release lifecycle fully valid.

================================================
LIVE FLEET STATUS
================================================
Active Missions: 0
Archived Missions: 0
------------------------------------------------
Drone: 105
Current Node: 0 (Warehouse_Alpha)
Battery Remaining: 30
Status: IDLE
------------------------------------------------
Drone: 104
Current Node: 2 (East_Corridor_1)
Battery Remaining: 86
Status: IDLE
------------------------------------------------
Drone: 103
Current Node: Pad 3
Battery Remaining: 22
Status: CHARGING
------------------------------------------------
Drone: 102
Current Node: 0 (Warehouse_Alpha)
Battery Remaining: 85
Status: IDLE
------------------------------------------------
Drone: 101
Current Node: 1 (North_Corridor_1)
Battery Remaining: 94
Status: IDLE
------------------------------------------------
```

```
================================================================================
STAGE 1: SYSTEM INITIALIZATION
================================================================================
[INFO] Loading static configurations and dynamic assets...

  * Fleet Status:          5 Active Drones (ID: 101, 102, 103, 104, 105)
  * Dispatch Depot:        1 Central Warehouse (Warehouse_Alpha at Node 0)
  * Charging Network:      3 Smart Charging Pads (Pad 1, Pad 2, Pad 3)
  * Airspace Restrictions: 2 No-Fly Zones (Restricted_Airport, Restricted_Hospital)

[Verification]: System initialization successful. All resources loaded.
================================================================================
```

Courier Dispatch Airspace Drone Matrix

DSA Project — Simulation Visualization Layer

Stage 1 — System Initialization

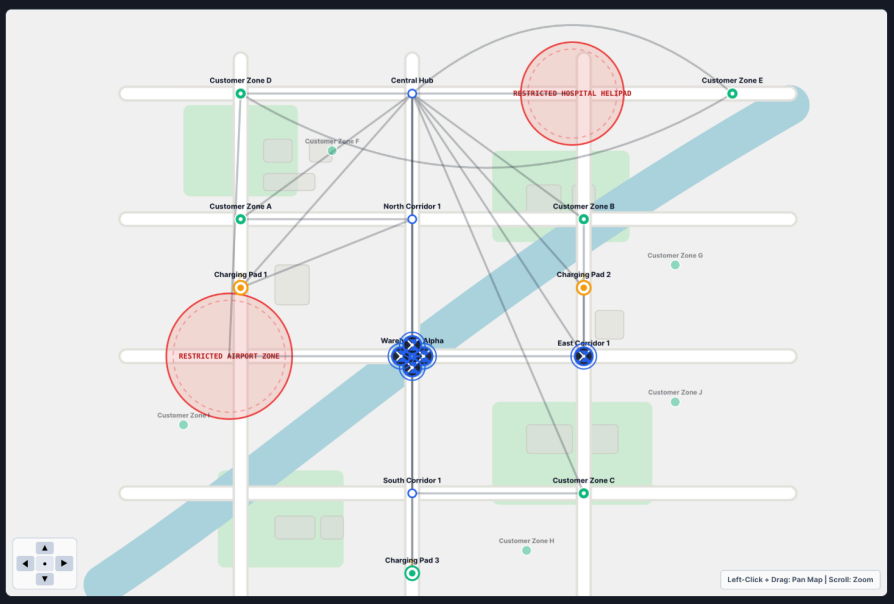
CORE DSA CONCEPT **Graph Topology & Node Mapping**

Constructing the 3D Airspace city map as an Adjacency List Graph containing Warehouse, Charging Pads, Corridor Hubs, Delivery Zones, and restricted No-Fly boundaries.

Action: Fleet loaded. 5 Drones at Warehouse (Node 0). 3 Charging Pads online. 2 No-Fly Zones active.

2D Airspace City Map (Google Maps Overlay)

Zoom Out | Zoom In | Center Map



Controls

STATE 1 / 17

▶ Run Demo

⏭ Next Tick

↺ Reset

- Battery ≥ 60%
- Battery 30–59%
- Battery < 30%
- Active Route
- Congested Corridor

Fleet Dashboard

Drone 101	IDLE	94%	NODE 0
Drone 102	IDLE	85%	NODE 0
Drone 103	IDLE	45%	NODE 0
Drone 104	IDLE	86%	NODE 2
Drone 105	IDLE	30%	NODE 0

Package Queue (FIFO)

PKG #1001

C++ Backend: Dijkstra · FIFO Queue · Min-Heap · Stack Checkpoints · Graph · HashMap · Congestion Recovery | Visualization Layer — React + SVG

Courier Dispatch Airspace Drone Matrix

DSA Project — Simulation Visualization Layer

Stage 7–8 — Storm Event + Stack Rollback (Undo)

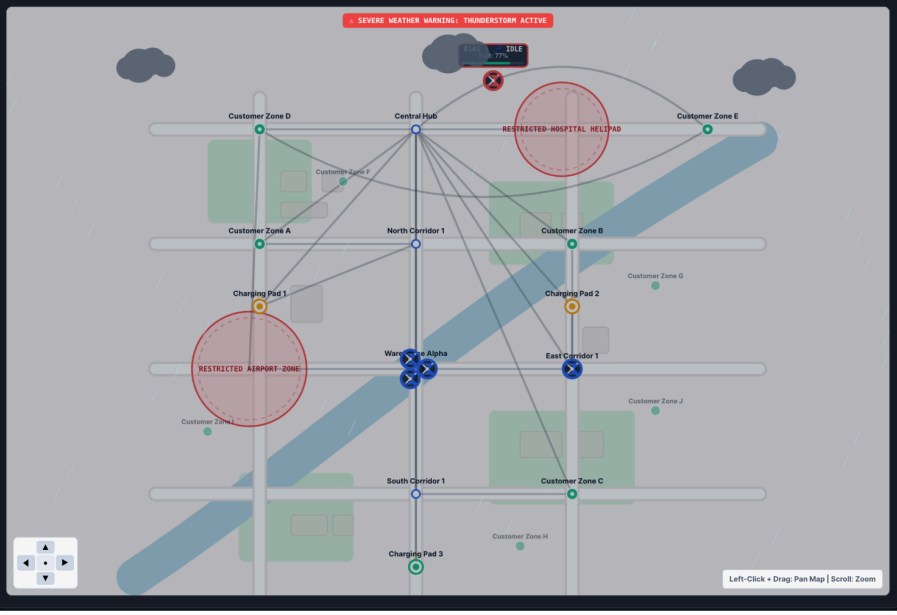
CORE DSA CONCEPT **Stack-based Flight State Rollback (Flight Undo)**

Sudden thunderstorm alert. Using Stack backtracking (std::stack) to pop historical safe coordinates, reverting the drone's position to its last safe checkpoint.

Action: Thunderstorm triggered. Drone 101 emergency halted. Stack pops last checkpoint → Node 10.

2D Airspace City Map (Google Maps Overlay)

Zoom Out | Zoom In | Center Map



Controls

STATE 6 / 17

⏸ Running...

⏭ Next Tick

↺ Reset

- Battery ≥ 60%
- Battery 30–59%
- Battery < 30%
- Active Route
- Congested Corridor

Fleet Dashboard

Drone 101	EMERGENCY	77%	NODE 10
Drone 102	IDLE	85%	NODE 0
Drone 103	IDLE	45%	NODE 0
Drone 104	IDLE	86%	NODE 2
Drone 105	IDLE	30%	NODE 0

Package Queue (FIFO)

PKG #1001

Future Enhancements

- 3D Airspace Engine
- Real GPS Coordinates
- Live Weather Integration
- Real Drone Telemetry
- WebSocket Communication
- Database Persistence
- Multi-City Support
- AI-Based Route Prediction

Conclusion

Drone-Matrix successfully demonstrates the practical application of Data Structures and Algorithms in solving large-scale drone logistics problems. The system integrates Graphs, Queues, Stacks, HashMaps, Priority Queues, and shortest-path algorithms to manage routing, congestion, battery safety, emergency recovery, and charging operations. The project meets all major problem-statement requirements while providing a scalable architecture suitable for future expansion.

References

1. C++ STL Documentation
2. Dijkstra Shortest Path Algorithm
3. React Documentation
4. Vite Documentation
5. Graph Theory References
6. Operating Principles of Autonomous Drone Networks

GitHub Repository

Repository Link: https://github.com/Samarth-ITM/Drone-Matrix

Demonstration Link

Visualization URL: https://drone-martix.vercel.app/